SWGL -- Adding a character or a faction to the dynamic character menu

2026-10-13

What the dynamic menu is made of

The character menu has no hard-coded character. At startup the game reads every `ext_data/characters/*.fac` (factions) and `ext_data/characters/*.cha` (characters) from all the pk3 and builds the lists. A new character or faction is therefore **data only**: no code change, no menu change. Put the files in a `Z_*.pk3` (loaded after the `SWGL_*.pk3`). The stock characters are in `SWGL_Characters.pk3`.

File (example)	Role	Required
`ext_data/characters/elween.cha`	Character: code, name, icon, factions, tags, variants. Any file name; one file can hold many characters.	Yes
`models/players/elween/model.glm` + `model_default.skin` (+ `sounds.cfg`, textures)	The player model of each variant (`model` field).	Yes
`models/players/elween/icon_default.tga` (or `.png` / `.jpg`)	Icon of the character and of each variant.	Yes
`ext_data/characters_conf/INF_ELWEEN_DEFAULT_def.cfg`	Default configuration of a variant: stats, force powers, weapons, saber, team. One per variant.	Yes (see section 4)
`ext_data/npcs/elween.npc`	NPC used when the character is spawned as an NPC (`npcName`).	Only for NPC mode
`strings/English/charelween.str`	Display name (`nameKey`) and biography (`descriptionKey`).	Recommended
`ext_data/characters/myfaction.fac`	New faction(s).	Only for a new faction
`strings/English/swgl_factions.str`	Faction names, if the faction uses the `swgl_factions_` keys.	Only for a new faction

How a click works: the menu sets `g_charKey` (character code) and `ui_variant_code` (variant code), then loads `ext_data/characters_conf/<g_charKey>_<ui_variant_code>_def.cfg`. Everything the player edits (stats, powers, weapons, saber) lives in that cfg and in the saved configurations, not in the `.cha`.

1. Add a character

1.1 The model

A variant points at a normal player model folder, `models/players/<model>/`, with `model.glm` and a `model_default.skin` (see the stock models; the menu preview and the skin list are built from the `.skin` files of the folder). The `.skin` files and textures follow the usual Jedi Academy rules. Nothing special is needed for the menu except the icon.

The **icon** is any image the renderer can register as a shader (path without extension is fine): `models/players/elween/icon_default.tga`. It is drawn in a 90 x 90 square in the grid and in a small square for the variants. Use a square image; the portrait tool (`charportraits` console command, `cg_characters.cpp`) renders one for every skin of every variant model into `portraits/<model>/<skin>.tga` if you do not want to paint it.

1.2 The `.cha` file

Minimal character, one variant:

```
{
    code            INF_ELWEEN
    name            "Elween Beli"
    icon            models/players/elween/icon_default.tga
    factions        REB,GE
    tags            CT_JEDI,CT_HERO

    Variant
    {
        code                        DEFAULT
        name                        "Elween Beli"
        icon                        models/players/elween/icon_default.tga
        model                       elween
        npcName                     elween
        descriptionKey      charelween_DESC
    }
}
```

Character fields:

Field	Meaning
`code`	**Required.** Unique id, max 31 characters, no spaces. Convention: `<FACTION>_<NAME>`. It is part of the cfg file names and of the saved-configuration folders, so **do not rename it later**. A block without a `code` is ignored with a warning.
`name "..."`	Plain text name. Shown in the grid. Use it, or `nameKey`, or neither (the code is shown).
`nameKey`	String reference (see section 5) used when `name` is empty. Also searched by the name filter.
`icon`	Icon of the grid square.
`factions`	Faction codes separated by commas, max 255 characters, e.g. `REB,GE`. A character can be in several.
`tags`	Tag codes separated by commas, e.g. `CT_JEDI,CT_HERO`. `none` means no tag. Tags need no declaration: every distinct tag of every character becomes an entry of the Tags screen, sorted by code.

Variant fields (`Variant { ... }`, as many as you want; a page shows 4 and has arrows):

Field	Meaning
`code`	**Required.** Id of the variant, max 31 characters (`DEFAULT`, `CW`...). Part of the cfg name. A variant without code is not shown.
`name "..."`	Title shown on the character screen. Order of use: variant `name`, then character `name`, then character `nameKey`, then code.
`icon`	Icon of the variant square.
`model`	Folder name under `models/players/`.
`npcName`	NPC type (a name from an `ext_data/npcs/*.npc` file) used in NPC mode. Sets `ui_npc_type`; the cfg can also set it (see 4).
`descriptionKey`	Biography string reference, shown in the Bio box. Written without `@`.
`npcRandom`, `skinRandom`, `saberRandom`, `saberDualRandom`, `saberStaffRandom`, `saberColorRandom`	Random parts, drawn at each NPC spawn and each Apply (`g_characterRandom.cpp`). Lists look like `saberRandom { "kit_fisto" "plo_koon" }`.

Weapons and powers come from the `_def.cfg`.

The file is a normal Quake-style token file: `//` comments, tabs or spaces, a block ends at the matching `}`. The limit is `MAX_CHARACTERS` (2048) characters in total over all pk3 (a message is printed when it is exceeded).

1.3 The default configuration (`_def.cfg`)

Create `ext_data/characters_conf/<CHARACTER CODE>_<VARIANT CODE>_def.cfg` for every variant, e.g. `INF_ELWEEN_DEFAULT_def.cfg` and `INF_ELWEEN_WARRIOR_def.cfg`. The name is compared case-insensitively, but the code and the variant code must match **exactly**, including underscores (a `-` in the code does not match a `_` in the file). The easiest way is to copy the cfg of a similar stock character and edit it. A cfg is a list of `cvar value` lines:

```
ui_health 200
ui_team enemy                  // player, enemy, ...
ui_char_color_red 255          // model colour
ui_char_color_green 255
ui_char_color_blue 255
ui_jump_level 2                // force powers 0-3: ui_push_level, ui_pull_level, ui_speed_level,
ui_push_level 3                //   ui_sense_level, ui_absorb_level, ui_heal_level,
ui_protect_level,
ui_heal_level 2                //   ui_mindtrick_level, ui_stasis_level, ui_blast_level,
ui_grasp_level,
ui_grip_level 3                //   ui_grip_level, ui_lightning_level, ui_drain_level,
ui_rage_level,
ui_lightning_level 0           //   ui_destruction_level, ui_fear_level, ui_strike_level,
                               //   ui_sabthrow_level, ui_saboff_level, ui_sabdef_level
ui_weaponOne weapon_boba       // up to six weapon classes: ui_weaponOne ... ui_weaponSix
(weapon_none = empty)
ui_weaponTwo weapon_disruptor
ui_npc_weapon weapon_boba      // weapon of the NPC
ui_saber empty                 // saber name from sabers.sab, or empty for none
ui_saber_color blue
ui_saber2 empty
ui_saber2_color red
ui_saber_styles 0
ui_lightning_color blue
ui_npc_type elween             // NPC type; overrides npcName
ui_force 100
```

Copy a complete stock cfg (e.g. `BH_Boba_default_def.cfg`) so that every line is present: a missing line keeps the value of the **previously selected character**.

If the cfg is missing the game looks for `ext_data/characters/default.cfg` (which does not exist in the stock pk3s), so the character would inherit the previous one's stats and the console prints `no character cfg found for code ... variant ...`.

Saved loadouts are separate: the player's *Save* writes `characters_configs/<CODE>_<VARIANT>/<name>.cfg` and appears in "My Characters". They are never part of a pk3.

1.4 The NPC configuration

The `.npc` file is mainly used by the **command line** (the `npc spawn <name>` console command, map scripts and maps). It describes a complete NPC on its own: model, weapon, saber, health, force powers, AI and sounds. When you spawn an NPC from the character screen, everything you set on that screen (model and skin, team, weapon, saber and colours, health, force powers) **replaces** the same values written in the `.npc`. The `.npc` still gives the settings the screen does not have: the AI skills, the speeds, the sounds, the size and a few more (list below).

The name in `npcName` (or the `ui_npc_type` line of the `_def.cfg`, which wins if both exist) must be the name of a block of an `ext_data/npcs/*.npc` file. Without a matching block the character is still playable normally, but the NPC spawn from the screen does nothing (the console shows `ERROR: Couldn't spawn NPC <name>`).

The exception is a variant with `npcRandom`: the NPC is picked from that list at each spawn, so it needs no `npcName` (every name in the list must exist).

The easiest way is to copy a stock NPC of the same kind and change it. Example (`ext_data/npcs/elween.npc`):

```
elween                              // name of the NPC = what you write in npcName
{
    fullName                "Elween"
    playerModel             elween
    rank                    commander
    playerTeam              TEAM_ENEMY
    enemyTeam               TEAM_PLAYER
    class                           CLASS_REBORN
    sex                             female
    yawSpeed                110
    walkSpeed               55
    runSpeed                200
    snd                             imperial_hf2
    sndcombat               imperial_hf2
    sndextra                imperial_hf2
    sndjedi                         imperial_hf2
    weapon                          WP_SABER
    saber                           asajj_ventress_bh
    saberColor              purple
    saberStyle              1
    saberStyle              2
    health                          200
    FP_HEAL                         2
    FP_SPEED                2
    FP_GRIP                         3
    FP_SABER_DEFENSE 2
    FP_SABER_OFFENSE 2
    reactions               3
    aim                             3
    move                    5
    aggression              3
    evasion                         4
    intelligence   5
    hfov                    160
    vfov                    160
}
```

The block starts with the NPC name on its own line, then `{ ... }`. One field per line, name then value; `//` starts a comment. A file can hold many NPCs.

Field	Meaning	Replaced by the character screen?
`fullName`	Name shown for the NPC (text).	No

Field	Meaning	Replaced by the character screen?
`playerModel`	Model folder under `models/players/`. `headModel`, `torsoModel`, `legsModel` can split the model in three parts.	Yes (model and skins)
`customSkin`	Skin of the model.	Yes
`playerTeam` / `enemyTeam`	Side of the NPC and of its enemies: `TEAM_PLAYER`, `TEAM_ENEMY`, `TEAM_NEUTRAL`.	`playerTeam` yes
`class`	Behaviour family (`CLASS_REBORN`, `CLASS_JEDI`, `CLASS_STORMTROOPER`, `CLASS_IMPERIAL`...). It drives which AI the NPC uses.	No
`rank`	Rank (`civilian`, `crewman`, `ensign`, `lieutenant`, `commander`...).	No
`sex`	`male`, `female` (voice and behaviour).	No
`weapon`	Weapon class (`WP_SABER`, `WP_BLASTER`...). Several lines give several weapons.	Yes
`saber`, `saberColor`, `saber2`, `saber2Color`	Saber hilt names (from `sabers.sab`) and colours.	Yes
`saberStyle`	Fighting styles allowed (1 fast, 2 medium, 3 strong).	Yes
`health`	Hit points.	Yes
`FP_...`	Force power levels, `FP_PUSH 3`, `FP_SABER_OFFENSE 2`... A power not listed is 0.	Yes
`aggression`, `aim`, `evasion`, `intelligence`, `move`, `reactions`, `vigilance`	AI skills, 1 (weak) to 5 (best).	No
`hfov`, `vfov`, `visrange`, `earshot`, `shootDistance`	How far and wide the NPC sees and hears and shoots.	No
`walkSpeed`, `runSpeed`, `yawSpeed`, `acceleration`	Speeds.	No
`snd`, `sndcombat`, `sndextra`, `sndjedi`	Voice folders under `sound/chars/`.	No
`scale`, `width`, `height`, `crouchheight`	Size of the NPC.	No
`animOverride`	Replace the animation set of the model.	No

2. Add a faction

A faction is a block in a `.fac` file. Create `ext_data/characters/myfaction.fac`:

```
Faction
{
    code            mymod
    nameKey                 myfaction_mymod
    icon            gfx/factions/mymod.tga
}
```

Field	Meaning
`code`	Faction code, max 15 characters. This is what the `factions` field of a character lists. Case-insensitive.
`nameKey`	String reference of the name shown in the faction list. If the string is not found, the raw key is shown. The list is **sorted by this raw key**, not by the displayed text.
`icon`	Not drawn by the current menu (the faction column is a text list since commit `3e77773c`). Keep it for compatibility; the stock icons are `gfx/factions/<code>.tga`.

Then give the faction its name in `strings/English/myfaction.str`:

```
REFERENCE                   MYMOD
LANG_ENGLISH        "My Faction"

ENDMARKER
```

and use `factions MYMOD` in the `.cha` of your characters. The stock factions are in `ext_data/characters/swgl.fac` of `SWGL_Characters.pk3` (41 entries; names in `strings/English/swgl_factions.str`): tor, drse, rse, ase, bos, rot, hr, rep, cis, reb, nr, ga, ge, ir, fe, dkge, res, fo, se, bh, hc, bs, cd, crim, zc, cv, ewok, yv, mando, dw, skir, mnc, ns, rev, ea, ee, jo, so, ufu, crtrs, inf.

3. Tags

A tag is just a word in the `tags` of a character. The Tags screen lists every distinct tag, sorted by code, and a character is shown if it has **at least one** selected tag. To show a translated name, add a string `SWGL_TAGS_<CODE>` (a file `swgl_tags.str` with `REFERENCE <CODE>`); there is none in the stock pk3, so the raw code is displayed. The maximum is 256 tags.

4. Names and biographies (`.str`)

A string reference is `<file name>_<REFERENCE>`, case-insensitive, where the file is `strings/English/<file name>.str` (one folder per language, `English`, `French`...). For `nameKey charelween_NAME` the game reads `charelween.str`:

```
REFERENCE                   NAME
LANG_ENGLISH        "Elween Beli"

REFERENCE                   DESC
LANG_ENGLISH        "Elween Beli, Hand of the emperor."

ENDMARKER
```

`descriptionKey charelween_DESC` is shown in the Bio box. Use quotes around the text; write `\n` for a new line. The simplest option is to skip `nameKey` and put `name "..."` in the `.cha`, like the stock characters do; keep `descriptionKey` for the biography. The `swgl_characters.str` file of `SWGL_Characters.pk3` holds some stock names.

Search and sort use the raw text of `name` / `nameKey` / `code`, not the translation. The grid is sorted by `nameKey` if there is one, else `name`, else `code`: a key like `charelween_NAME` sorts under "c", not under "E". Prefer `name` if the alphabetical place matters.

5. Pack and test

1. Build a pk3 (zip) whose paths start at the game root: `ext_data/characters/elween.cha`, `ext_data/characters_conf/...`, `ext_data/npcs/...`, `models/players/elween/...`, `strings/English/...`. Name it `Z_<name>.pk3` and put it in `GameData\SWGL\`. It is loaded after every `SWGL_*.pk3`, so it can also override a stock file (same path).
2. Start the game and read the console. Look for:
   - `Parsing elween.cha` (the file is read),
   - `WARNING: character without a code in a .cha file, ignored`,
   - `Error: Invalid syntax, expected "{"`,
   - `no character cfg found for code '...' variant '...'` when you select the variant (missing or misnamed `_def.cfg`).
3. Open the character menu. Select your faction, then your character, then each variant. Check: name and icon in the grid, the title, the Bio, the 3D preview and the skins list, the stats / powers / weapons tabs (they must show your cfg values), then Apply, and Spawn in NPC mode.
4. `characterStat <CODE> [<VARIANT>]` (console, cheats on) prints what the game loaded for a character and its variants, to check that your `.cha` was read as you wrote it.

6. Checklist and common mistakes

Symptom	Cause
Character missing from the menu	No `code`, the `.cha` is not in `ext_data/characters/`, a syntax error (missing brace) earlier in the same file, or a faction / tag filter is selected.
Character shown only under the wrong faction filter	Faction code typed differently from the `.fac` `code`, or a substring clash (`SE` / `RSE`).
Stats, weapons and powers are those of the previous character	`_def.cfg` missing or misnamed (the code or variant code does not match), or the cfg does not contain every line.
Name shows as `CHARELWEEN_NAME` or the code	The `.str` file name does not match the key prefix, the `REFERENCE` is misspelt, or the file has no `ENDMARKER`.
Empty square instead of the icon	Icon path wrong or the image is not in the pk3; paths use `/` or `\` but no leading slash.
Variant not shown	Variant has no `code`.
NPC does not spawn	`npcName` / `ui_npc_type` is not the name of a block in an `.npc` file.
Renamed a character code and the saved configs vanished	Saved configs are stored in `characters_configs/<CODE>_<VARIANT>/`; they are tied to the code.
New faction icon not visible	Expected: the faction column is text only.

Code references: `g_charactersLoad.cpp` (parsing), `g_characters.h` (structures and limits), `cg_characters.cpp` (menu, filters, grid), `ui_main.cpp` `UI_LoadCharacterCfg` (cfg lookup order: `ext_data/characters/<code>_<variant>[_NPC].cfg`, then `ext_data/characters_conf/<code>_<variant>_def.cfg`, then `ext_data/characters/default.cfg`).